

8-3 · 수집 → Lake → Spark → VectorDB 흐름 자동화 · 실습 1.5H

수집 → Lake → 변환 태스크 구현

같은 네트워크에 Qdrant, 그 위에 세 태스크

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① Qdrant 를 Airflow 와 같은 compose 에 추가하고 서비스 이름으로 접근한다
- ② qdrant-client·openai 패키지와 OPENAI_API_KEY 를 컨테이너에 전달한다
- ③ collect·transform·validate 세 태스크를 rag_pipeline 에 배선해 초록으로 만든다
- ④ 나쁜 문서를 일부러 넣어 validate 가 아래를 막는 것을 확인한다

사전 준비: 8-1 Airflow 기동(docker compose ps) · OpenAI 키 · 8-2 분해안 · 따로 띄운 Qdrant 는 내린다

일곱 단계 — ~/airflow-lab 에서 끝까지

2단계(같은 네트워크)를 통과하지 못하면 뒤가 전부 실패합니다 — 반드시 먼저 통과시키세요

1. Qdrant 추가	2. 네트워크	3. 패키지·키	4·5. 태스크	6·7. 배선·실패
기존 내리고 compose 에	http://qdrant:6333 → 200	.env 두 줄 yaml 한 줄	collect·transform validate	trigger → 「짧음」 넣기

Qdrant 를 compose 에 붙이고 두 곳에서 부른다

기존 내리기	<code>docker stop qdrant && docker rm qdrant</code>
compose 추가	services: 아래 qdrant 블록 — 6333:6333 · ./qdrant_storage
기동	<code>docker compose up -d qdrant</code>
호스트에서	<code>curl -s http://localhost:6333/collections → {"result":{"collections":[]}}...</code>
컨테이너 안에서	<code>docker compose exec airflow-scheduler python -c "... urlopen('http://qdrant:6333/collections').status" → 200</code>

네트워크 목록으로도 확인: `docker network inspect airflow-lab_default` — 결과에 qdrant 가 함께 있어야 한다

패키지와 API 키를 컨테이너 안까지

1 **.env 에 두 줄 — 패키지 목록과 OPENAI_API_KEY=sk-본인키**
_PIP_ADDITIONAL_REQUIREMENTS=qdrant-client openai

2 **docker-compose.yaml 에 한 줄**
x-airflow-common 의 environment: 에 OPENAI_API_KEY: \${OPENAI_API_KEY:-}

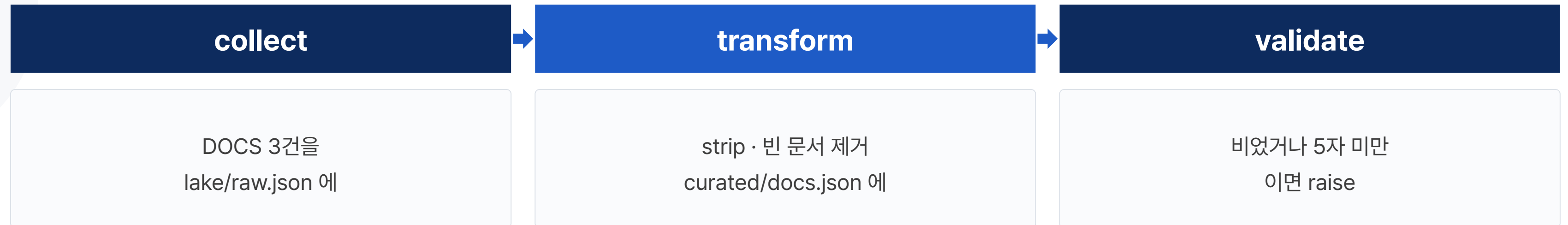
3 **up -d 로 재생성 — restart 로는 반영되지 않는다**
python -c "import qdrant_client, openai; print('ok')" · env | grep OPENAI

먼저 할 것

**.env 를
.gitignore 에**

| **_PIP_ADDITIONAL_REQUIREMENTS** 는 학습용 편의 기능 — 시작할 때마다 설치가 다시 돈다. 운영은 커스텀 이미지 빌드가 정석

세 함수 — 같은 경로에 덮어쓴다



BASE = /opt/airflow/dags/data — dags/ 마운트 안의 실습용 저장소 · validate 는 건수만 return(작은 값이라 XCom 가능)

몇 번을 재시도해도 결과가 같다 — 8-2 의 맥등이 코드로 나타난 것 · 검증을 따로 두면 실패 지점이 화면에 그대로 드러난다

배선하고 돌려 본다 — 세 태스크가 초록

DAG 설정	schedule=None · catchup=False · retries 1 · retry_delay 30초
배선	t_collect >> t_transform >> t_validate
임포트 오류	\$CLI dags list-import-errors
활성화	\$CLI dags unpause rag_pipeline
실행	\$CLI dags trigger rag_pipeline

CLI="docker compose exec airflow-scheduler airflow" — 한꺼번에 만들면 어디가 틀렸는지 못 찾는다, 여기서 확실히 통과

검증이 실제로 막는지 — 「짧음」을 섞어 본다

검사기는 일부러 깨뜨려 봐야 동작한다고 말할 수 있습니다

DOCS 수정	다시 trigger	validate 빨강	아래는 시작 안 함	원래대로
두 번째에 "짧음" (5자 미만)	\$CLI dags trigger rag_pipeline	「너무 짧은 문서가 섞여 있습니다」	나쁜 데이터가 내려가지 않는다	DOCS 되돌리고 저장

이 1.5시간이 끝날 때 확인할 것

- 1 `curl localhost:6333/collections` 응답이 JSON 이다
- 2 컨테이너 안에서 `http://qdrant:6333` 이 200 이다
- 3 `import qdrant_client, openai` 가 ok · env 에 `OPENAI_API_KEY` 가 보인다
- 4 `rag_pipeline` 의 `collect` → `transform` → `validate` 가 초록이다
- 5 「짧음」을 넣으면 `validate` 가 빨강이 되고 아래는 시작하지 않는다

8-3 · 실습 1.5H 마무리

다음은 임베딩 → VectorDB 적재 (1H)

embed_and_load 를 붙여 네 태스크를 끝까지 돌리고, 두 번 돌려 먹등을 확인한 뒤 @daily 스케줄을 겁니다

✓ OpenAI 임베딩 → Qdrant lecture_docs 적재

✓ points_count 3 확인

✓ 두 번 돌려 개수가 그대로인지

✓ SparkSubmitOperator 구조 확인

가져갈 것

초록이 된 세 태스크 · 되돌린 DOCS